

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GXESL208 IT WORKSHOP

LAB MANUAL

PREPARED BY:

Ms REMYA K R

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.

M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.

M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

M1: To seek and strive for advancements in technological domains.

M2: To create socially responsible engineers in the field of computer science.

M3: To bridge the gap between science and mankind through meaningful innovation and service.

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in the IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable

development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

IT WORKSHOP

Program: B. Tech	Department: Computer Science and Engineering
Course: IT Workshop	Semester: S2 Batch: C Credits: 1
Course Code: GXESL208	Regulation: 2024
Course Area/Stream:	L: T: TW/SL: P: 2
Corresponding Lab Course Code (If Any): Nil	Lab Course Name:

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Experiment with the fundamental hardware components of a computer and how to interface them with software systems.
CO2	Make use of the command line of Linux operating system and shell Programming.
CO3	Experiment with the data network communication scenarios using Wireshark.
CO4	Develop basic websites using HTML, CSS & JavaScript and manage the versions.

CO - PO/PSO MAPPING: (Copy and paste values from competency matrix)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	3		3						3		1	2
CO2	3	3	3	3	3						3		2	3
CO3	3	3	3	3							3		1	3
CO4	3	3	3	3	3						3		2	3

CONTENTS

Exp No	Exp Name	Page No
1	Getting started with Basics of Computer Hardware	6
2	Familiarization of Boot Process	13
3	Familiarizing installation of Linux and Windows operating systems	16
4	Familiarizing Basic Unix/Linux commands	19
5	Familiarizing Computer network components	26
6	Familiarizing Basic Networking commands	31
7	View network traffic using Wireshark/Packet tracer	35
8	Familiarizing the steps how to configure and establishing a network connecting	39
9	Shell programming in Linux(bash)	43
10	Form validation using Javascript	51
11	Image Slider using HTML,CSS and JavaScript	59
12	Familiarisation of LaTeX	64
13	Familiarisation of Development Environments	70
14	Introducing repositories- Git/Bitbucket	77

EXP NO:1**Getting started with Basis of Computer Hardware****AIM**

To understand the basic components of computer hardware, their functions, and how they interact to form a complete computer system.

LEARNING OUTCOMES

To demonstrate the fundamental concepts of basic components of computer hardware and their functions.

DESCRIPTION

The important computer hardwares of an operating system are:

CPU Box

CPU box familiarization is about understanding the key components inside a computer, such as the motherboard, processor, RAM, storage drives (HDD/SSD), PSU, GPU, and cooling systems, along with their functions and interconnections. It covers identifying input/output ports like USB, HDMI, and Ethernet, understanding the role of power management, learning safe assembly/disassembly practices, and proper handling of sensitive components like the CPU and RAM. Additionally, it includes recognizing the importance of cooling and airflow, managing cables effectively, and troubleshooting basic hardware issues, providing a strong foundation for understanding computer hardware and its practical applications.



Figure 1: CPU

Motherboard

The motherboard is the main circuit board of a computer that acts as a central hub, connecting all components and allowing them to communicate. It hosts critical hardware such as the CPU, RAM, storage devices, and GPU, and includes ports for peripherals and expansion cards. Key features of a motherboard include:

- CPU Socket: Slot for installing the processor.
- RAM Slots (DIMM): Slots for memory modules.
- Chipset: Manages data flow between the CPU, memory, and peripherals.
- Expansion Slots (PCIe): For GPUs, sound cards, or other add-ons.
- Storage Interfaces: SATA or M.2 ports for connecting HDDs and SSDs.
- I/O Ports: USB, HDMI, Ethernet, and audio ports for external devices.
- Power Connectors: Connect to the PSU for powering the board and attached components.



Figure 2: Motherboard

CPU & Chipset

The CPU is the brain of the computer, responsible for executing instructions and performing computations. It fits into the CPU socket on the motherboard, which varies by processor type and brand (e.g., Intel or AMD). Common components of a CPU include:

- Cores: Independent processing units for multitasking.
- Threads: Virtual cores for handling multiple tasks simultaneously.
- Cache: High-speed memory within the CPU for quick data access.



Figure 3: Chipset

Chipset:

- The chipset is a set of integrated circuits on the motherboard that manages communication between the CPU, memory, storage devices, and peripherals. It acts as the motherboard's traffic controller, ensuring efficient data flow between components.

Modern chipsets are divided into:

- Northbridge: (Integrated into the CPU in newer systems) Handles high-speed communication between the CPU, RAM, and GPU.
- Southbridge: Manages slower peripherals like USB, SATA, and audio.
- The chipset determines key motherboard features, such as:
 - Supported CPU models and RAM types.
 - Number of expansion slots and USB ports.
 - Overclocking capabilities.
 - Both the CPU and chipset are critical for overall system performance and compatibility with other hardware.

Interface Cards

- Add-on cards installed in expansion slots to enhance or add functionalities.
- Examples: Graphics Cards (GPU), Sound Cards, Network Interface Cards (NIC), TV Tuner Cards.
- Interface: PCIe (Peripheral Component Interconnect Express) slots for modern cards.

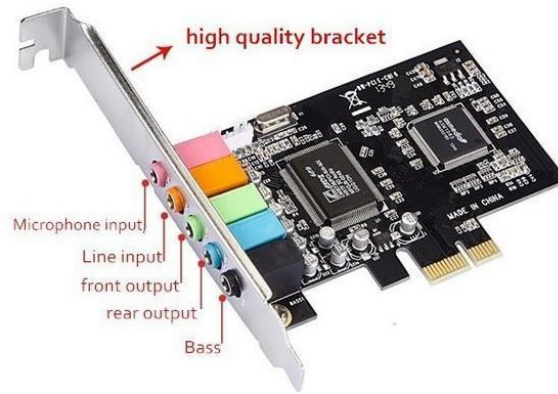


Figure 4: Sound Card

Card Slots

- Slots on the motherboard for attaching interface cards.
- PCIe Slots: High-speed slots for GPUs and other modern interface cards.
- PCI Slots: Older standard for expansion cards.
- M.2 Slots: Used for high-speed SSDs and wireless adapters.

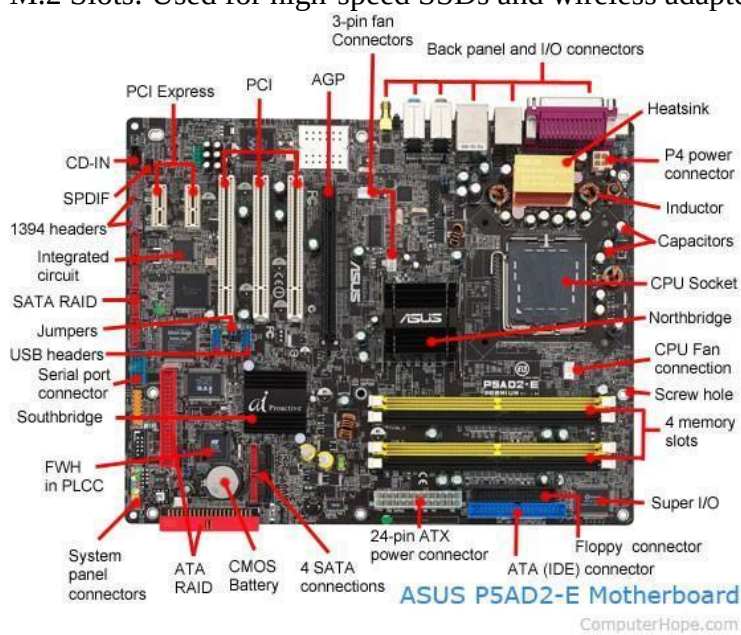


Figure 5: Slots on Motherboard



Figure 6: HDD



Figure 7: SSD

Hard Disk

- Primary storage device for data and operating systems.
- Types: HDD (Hard Disk Drive): Mechanical drive offering large capacity at lower speeds.
- SSD (Solid State Drive): Faster, more reliable, and energy-efficient, with no moving parts.
- Connected via SATA (Serial ATA) or NVMe (Non-Volatile Memory Express) interfaces.

Cables

- Essential for connecting components and peripherals.
- Power Cables: From the SMPS to components like the motherboard, GPU, and storage.
- Data Cables: SATA cables for storage drives, ribbon cables for legacy devices.
- Interface Cables: HDMI, VGA, USB, and Ethernet for connecting peripherals and external devices.

SMPS (Switched Mode Power Supply)

- Converts AC power from the outlet into DC power for the computer.
- Supplies power to the motherboard, CPU, GPU, storage devices, and peripherals.
- Includes connectors such as 24-pin ATX (motherboard) and 6/8-pin PCIe (GPU).



Figure 8: SMPS

NIC (Network Interface Card)

- Enables wired or wireless networking by connecting the computer to a network.
- Ethernet NIC: Wired connection via an RJ45 port.
- Wi-Fi Adapter: Wireless connection via PCIe or USB.
- NICs can be integrated into the motherboard or added as dedicated cards for advanced features.

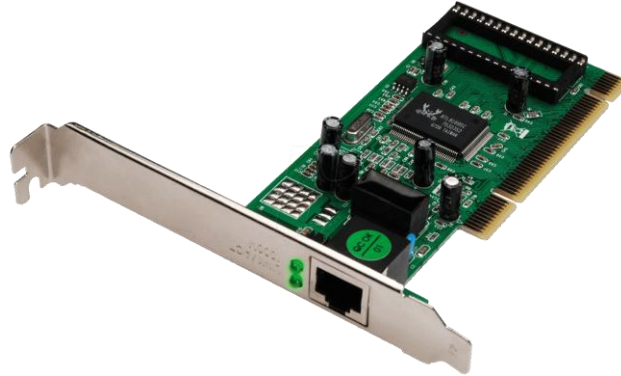


Figure 9: NIC Card

Various Ports

- Located on the motherboard and case for connecting peripherals.
- USB Ports: Universal interface for devices like keyboards, mice, and storage.
- Audio Ports: For headphones, microphones, and speakers.
- Video Ports: HDMI, DisplayPort, VGA, or DVI for monitors.
- Ethernet Port: For network cables.
- Power Port: For SMPS connection.

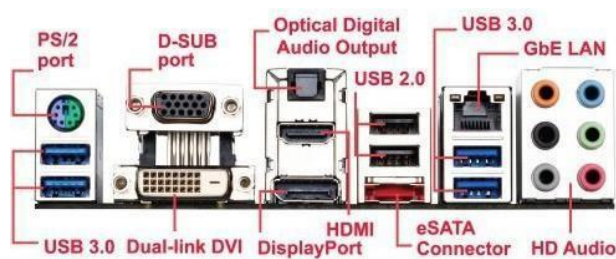


Figure 10: Ports on Motherboard

PROCEDURE

Open the Case: Remove the side panel(s) of the computer case.

Locate Major Components: Visually identify and physically locate the following components:

a) **Motherboard**

b) **CPU** (often under a cooler/heatsink)

- c) **RAM** modules (in slots)
- d) **GPU** (if present, in a PCIe slot)
- e) **Power Supply Unit (PSU)**
- f) **Storage Device(s)** (HDD/SSD)

Record Details: Record the make, model, and any visible specifications for the identified components (e.g., RAM type and size, storage interface like SATA or M.2).

Trace Connections: Trace the main **cables** and **connectors** (e.g., 24-pin ATX power, 8-pin CPU power, SATA data/power) to understand the power and data flow within the system

MODEL / VIVA QUESTIONS

1. What is the primary difference between **RAM** and a **Hard Disk Drive (HDD)/Solid State Drive (SSD)**?
2. What is the **Northbridge** (or its modern integrated equivalent) on the motherboard primarily responsible for?
3. What does "**Form Factor**" mean when referring to a motherboard or PSU?
4. What is a "**Cache**" and why is it essential for the CPU?
5. What is the **BIOS/UEFI**, and what is its role when you first press the power button?

RESULT

Gained knowledge about essential computer hardware components such as the CPU, RAM, motherboard, storage devices, input/output devices, and power supply, and understood their roles in the functioning of a computer system.

EXP NO:2**Familiarization of Boot Process****AIM**

To understand the boot process of a computer, including the sequence of steps that occur from powering on the system to loading the operating system.

LEARNING OUTCOMES

To understand the sequence of events a computer executes from the moment the power button is pressed until the operating system (OS) is ready for user interaction.

DESCRIPTION

The boot process is the sequence of events that occurs when a computer is powered on and prepares the system for use. The Process are:

1. Power-On Self-Test (POST):

- a. When the computer is turned on, the BIOS/UEFI firmware on the motherboard initiates a self-diagnostic test called POST.
- b. It checks the basic hardware components, such as the CPU, RAM, and storage devices, to ensure they are functioning correctly.
- c. If errors are found, the system displays error messages or beep codes.

2. Initialization of Firmware:

- a. The BIOS/UEFI firmware initializes hardware and configures the system for booting.
- b. It locates the boot device (e.g., HDD, SSD, USB drive) specified in the boot order settings.

3. Boot Loader Activation:

- a. The BIOS/UEFI hands over control to the boot loader located on the boot device.
- b. Common boot loaders include GRUB (Linux) or Windows Boot Manager.

4. Loading the Operating System (OS):

- a. The boot loader loads the kernel of the operating system into memory.

- b. The kernel initializes essential system components and manages resources like CPU, memory, and input/output devices.

5. User Interface Launch:

- a. Once the kernel has initialized, the OS starts system services, drivers, and the graphical user interface (GUI).
- b. The login screen or desktop environment is displayed, completing the boot process.

PROCEDURE

Preparation: Ensure the computer is completely powered off (cold boot). Connect only the necessary peripherals (monitor, keyboard, mouse) and the power cable.

Initiate Cold Boot: Press the power button. Immediately observe the following:

- **Power Indicators:** Note the time taken for any power LED lights to stabilize.
- **Fan Activity:** Note when the CPU and system fans begin to spin.
- **POST Screen:** Observe the initial screen display. Record the key (e.g., F2, Del, F10) required to enter the BIOS/UEFI setup.

Identify Beep Codes (If Applicable): If the system emits any audible beeps, record the pattern (e.g., one short, two long). Refer to the motherboard manual (or search online) to determine what a single, successful POST beep code means versus a sequence indicating an error.

OS Loading Handoff: Observe the point where the initial text/graphic screen disappears and the loading animation or screen for the Operating System (e.g., Windows logo) appears. This signifies the **Bootloader Handoff**.

MODEL VIVA QUESTIONS

1. Define the acronym POST. What is the primary goal of the POST stage?
2. If a computer emits a pattern of short beeps and does not display anything, what is the most probable component failure, and why?
3. Explain the significance of the **Boot Priority List** (Boot Order). What determines which device the firmware attempts to load first?

RESULT

Gained knowledge of the computer boot process, including stages such as POST (Power-On Self-Test), BIOS/UEFI initialization, bootloader execution, and OS loading, and understood how these steps work together to start a functional system.

EXP NO:3**Familiarizing installation of Linux and Windows operating systems****AIM**

To successfully install Windows and Linux on a personal computer using installation media (USB or DVD).

LEARNING OUTCOMES

To demonstrate the installation of operating systems (Windows/Linux).

SOFTWARE/HARDWARE REQUIRED:

Bootable USB/DVD

DESCRIPTION**Installing Windows**

- **Create Bootable Media:** Use tools like Rufus or Media Creation Tool to create a bootable USB or DVD with the Windows ISO file.
- **Boot from Installation Media:** Restart the system and enter the BIOS/UEFI settings (usually by pressing F2, F12, Del, or Esc during boot) and Set the boot order to prioritize the USB/DVD.
- **Windows Installation Steps**
 - Select your language, time format, and keyboard layout. Click "Next."
 - Click "Install Now."
 - Enter the product key or select "I don't have a product key."
 - Choose the installation type:
 - Upgrade: To keep existing files.
 - Custom: For a fresh installation.
 - Create/modify partitions as needed.
 - Select the partition to install Windows and click "Next."
 - Wait for the installation to complete and follow the on-screen instructions for initial setup.

Installing Linux (Eg. Ubuntu)

1. **Create Bootable Media:** Use tools like Balena Etcher to create a bootable USB with the Linux ISO file.

2. Boot from Installation Media:

- a. Restart the system and enter the BIOS/UEFI settings.
- b. Set the boot order to prioritize the USB/DVD.

3. Linux Installation Steps

1. Select "Install Ubuntu" from the boot menu.
2. Choose your preferred language and keyboard layout.
3. Select the installation type:
 - a. Normal Installation: Includes additional software.
 - b. Minimal Installation: Only essential tools.
4. Choose the partitioning method:
 - a. Erase Disk and Install: For fresh installations.
 - b. Something Else: For custom partitioning.
 - c. Create partitions (e.g., root /, swap, and home /home).
5. Click "Install Now" and confirm changes.
6. Set up user accounts and passwords.

PROCEDURE

1. Pre-Installation Preparation and Media Creation
2. Disk Management and Partitioning
3. Installation Execution and Customization
4. Dual-BootPost-Installation and Verification
5. Configuration and Management

MODEL VIVA QUESTIONS

1. What is GRUB, and what is its role when a user selects an operating system at startup?
2. What is a "bootable installation media," and name one tool used to create a bootable USB drive for a Linux distribution?
3. When setting up Linux, what are the three essential partitions you created, and what is the primary

function of the **Swap** partition?

4. Define the term "**Dual-Boot.**" ?

RESULT

The installation of both Windows and Linux was successfully completed and fully operational on the computer, ready for use.

EXP NO: 4**Familiarizing Basic Unix/Linux Commands****AIM**

To learn and practice basic Linux commands to understand file management, process control, and system navigation in a Linux environment.

LEARNING OUTCOMES

Students will be able to use basic Linux commands to navigate the file system, manage files and directories, control processes, and perform essential system operations in a Linux environment.

DESCRIPTION

Unix/Linux commands are essential for navigating and managing a system efficiently. Here's an overview of some fundamental commands with their purpose:

1. ls (List Directory Contents):
 - a. Displays files and directories in the current directory.
 - b. Example: ls -l (detailed view with permissions, size, and date). [h!]

Options	Description
-l	Long listing format of files and directories, one per line
-a	List all hidden files and directories started with '.'
-F	Add a '/' classification at the end of each Directory
-g	List all files and directories with the group name
-i	Print index number of each files and directories
-m	List all file and directories separated by comma ','
-n	List numeric UID and GID of Owner and Groups
-r	List all files and directories in reverse order
-R	Short list all directories
-t	Sorted by modified time, started with the newest file

Figure 11: ls syntax

```

zaheer@linuxhunter:~$ ls -il
total 38808
4105341 -rw-rw-r-- 1 zaheer zaheer 37000 Sep 22 12:48 -1.png
4105338 -rw-rw-r-- 1 zaheer zaheer 38985 Sep 22 12:13 30days.png
4105339 -rw-rw-r-- 1 zaheer zaheer 39964 Sep 22 12:27 5day.png
4104847 -rw-rw-r-- 1 zaheer zaheer 7900 Sep 8 02:56 ali.png
4197919 drwx----- 3 zaheer zaheer 4096 Sep 21 23:18 AnyDesk
4104994 -rw----- 1 zaheer zaheer 1559238 Sep 20 13:50 asd.mp4
4104852 -rw-rw-r-- 1 zaheer zaheer 23485 Sep 4 01:52 asd.png
4104876 -rw-rw-r-- 1 zaheer zaheer 3853 Sep 10 00:14 as.png
  
```

Figure 12: ls syntax

2. mkdir (Make Directory):

- a. Creates a new directory.
- b. Example: mkdir new folder. _

```

bosko@pnep:~$ mkdir -m777 DirM
bosko@pnep:~$ ls -l
total 153028
-rw-rw-r-- 1 bosko bosko      0 Jul 31 15:20 123.txt
-rwxrwxr-x 1 bosko bosko    96 Jul 31 14:56 basic-loop.sh
-rwxrwxr-x 1 bosko bosko   371 Aug  1 03:03 break-and-continue.sh
-rwxrwxr-x 1 bosko bosko   215 Aug  3 03:51 concatenate-with-for-loop.sh
drwxr-xr-x 2 bosko bosko  4096 Jun  8 07:54 Desktop
drwxrwxr-x 2 bosko bosko  4096 Dec  1 05:15 DirM
drwxr-xr-x 2 bosko bosko  4096 Jun  8 07:54 Documents
drwxr-xr-x 2 bosko bosko  4096 Aug  3 03:52 Downloads

```



Figure 13: mkdir

3. cp (Copy Files/Directories):

- a. Copies files or directories.
- b. Example: cp file.txt backup/ (copies file.txt to the backup folder).

```

administrator@GFG19566-LAPTOP:~/practice$ ls
a.txt  b.txt
administrator@GFG19566-LAPTOP:~/practice$ cat a.txt
hello welcome to gfg
administrator@GFG19566-LAPTOP:~/practice$ cat b.txt
hello
administrator@GFG19566-LAPTOP:~/practice$ cp -f a.txt b.txt
administrator@GFG19566-LAPTOP:~/practice$ cat b.txt
hello welcome to gfg

```

Figure 14: cp

4. mv (Move/Rename Files):

- a. Moves or renames files and directories.
- b. Example: mv file.txt new name.txt (renames file.txt to new name.txt). _

```

kbuzdar@ubuntu:~$ ls
Desktop Documents Downloads sample1.txt snap Test
kbuzdar@ubuntu:~$ mv sample1.txt ~/Documents/
kbuzdar@ubuntu:~$ ls
Desktop Documents Downloads snap Test
kbuzdar@ubuntu:~$ ls ~/Documents/
sample1.txt

```

Figure 15: mv

5. grep (Search for Patterns in Text):

- Searches for specific text in files or outputs.
- Example: `grep "keyword" file.txt`.

```
yesmin@fahmida-VirtualBox:~/code$ cat customers.txt
ID      Name      Email      Phone
11      Md. Abir  abir@gmail.com  +8801813462458
23      Riya Chakroborti  riya@gmail.com  +8801937864534
45      Minhaz Ali  ali@gmail.com  +8801190761212
56      Maliha Chowdhury  maliha@gmail.com  +8801820001980
79      Maruf Sarkar  maruf@gmail.com  +8801670908966
yesmin@fahmida-VirtualBox:~/code$ grep -E '9+018' customers.txt
yesmin@fahmida-VirtualBox:~/code$ grep -E '8+018' customers.txt
11      Md. Abir  abir@gmail.com  +8801813462458
56      Maliha Chowdhury  maliha@gmail.com  +8801820001980
yesmin@fahmida-VirtualBox:~/code$
```

Figure 16: grep

6. rmdir (Remove Directory):

- Deletes an empty directory.
- Example: `rmdir old folder.`

```
prateekjangid@prateekjangid: ~
prateekjangid@prateekjangid:~$ ls
Desktop Documents INFO Music Public Templates
DETAIL Downloads LINUX Pictures snap Videos
prateekjangid@prateekjangid:~$ rmdir LINUX INFO DETAIL
prateekjangid@prateekjangid:~$ ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
prateekjangid@prateekjangid:~$
```

Figure 17: rmdir

7. chmod (Change File Permissions):

- Modifies file permissions.
- Example: `chmod 755 script.sh` (gives read/write/execute permissions to the owner and read/execute permissions to others).

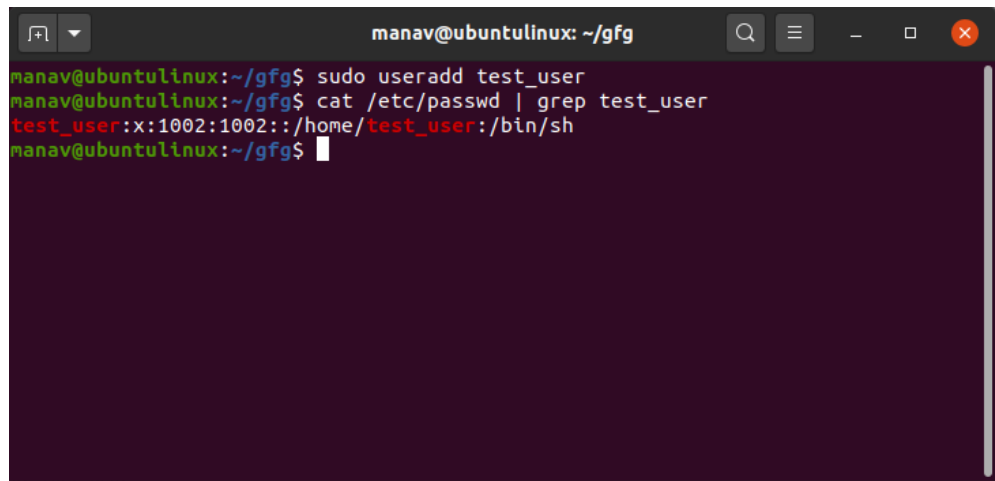
```
akash@ubuntu:~$ chmod g+r File2.txt
akash@ubuntu:~$ chmod g+w File2.txt
akash@ubuntu:~$ chmod g+x File2.txt
akash@ubuntu:~$ ls -l
total 68
drwxrwxr-x 4 akash akash 4096 Dec 14 12:21 Contents
-rw-rw-r-- 1 akash akash  229 Dec 14 12:21 Contents.zip
drwxr-xr-x 6 akash akash 4096 Nov 30 15:43 Desktop
drwxrwxr-x 2 akash akash 4096 Dec 14 11:41 Destination
drwxr-xr-x 2 akash akash 4096 Nov 30 11:59 Documents
drwxr-xr-x 2 akash akash 4096 Nov 24 11:33 Downloads
-rw-rw-rwx 1 akash akash  58 Dec 13 14:50 File1.txt
-rw-rwxr-- 1 akash akash  58 Dec 14 11:18 File2.txt
```

Permission granted

Figure 18: Chmod

8. useradd (Add New User):

- a. Creates a new user account.
- b. Example: `sudo useradd -m newuser` (creates a user with a home directory).

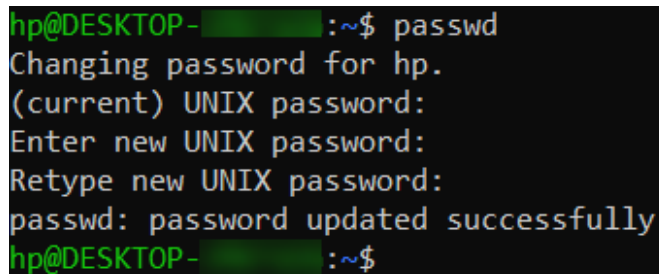


```
manav@ubuntuLinux: ~/gfg
manav@ubuntuLinux:~/gfg$ sudo useradd test_user
manav@ubuntuLinux:~/gfg$ cat /etc/passwd | grep test_user
test_user:x:1002:1002::/home/test_user:/bin/sh
manav@ubuntuLinux:~/gfg$
```

Figure 19: useradd

9. passwd (Change Password):

- a. Sets or changes a user's password.
- b. Example: `passwd username`.

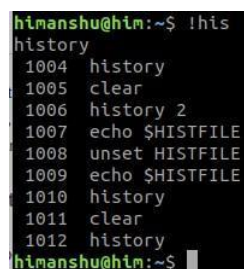


```
hp@DESKTOP-: ~$ passwd
Changing password for hp.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
hp@DESKTOP-: ~$
```

Figure 20: passwd

10. history (Command History):

- a. Displays a list of previously executed commands.
- b. Example: `history — grep mkdir` (searches for all mkdir commands in history).



```
himanshu@him:~$ !his
history
1004 history
1005 clear
1006 history 2
1007 echo $HISTFILE
1008 unset HISTFILE
1009 echo $HISTFILE
1010 history
1011 clear
1012 history
himanshu@him:~$
```

Figure 21: his

11. dmesg (System Boot and Kernel Messages):

- Displays messages generated by the kernel during boot or runtime.
- Example: `dmesg — grep error` (searches for errors in the kernel logs).

```
sara@pnep:~$ sudo dmesg -l info
[sudo] password for sara:
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-47-generic root=UUID=4c8c3a2e-121e-4bc7-a52a-f95665d4d4b6 ro quiet splash
[ 0.000000] KERNEL supported cpus:
[ 0.000000] Intel GenuineIntel
[ 0.000000] AMD AuthenticAMD
[ 0.000000] Hygon HygonGenuine
[ 0.000000] Centaur CentaurHauls
[ 0.000000] zhaoxin Shanghai
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x0000000000dfffff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000000dfff0000-0x0000000000dfffffff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fffc0000-0x00000000ffffffff] reserved
```

Figure 22: dmesg

12. cpuinfo (CPU Information):

- Displays detailed information about the CPU.
- Example: `cat /proc/cpuinfo`.

```
kali@kali: ~
GNU nano 6.4 /proc/cpuinfo
address sizes : 45 bits physical, 48 bits virtual
power management:

processor      : 1
vendor_id     : GenuineIntel
cpu family    : 6
model         : 61
model name    : Intel(R) Core(TM) i3-5005U CPU @ 2.00GHz
stepping      : 4
microcode     : 0xffffffff
cpu MHz       : 1995.379
Cache size    : 3072 KB
```

Figure 23: Cpuinfo

13. uname (System Information):

- Displays system information like kernel name, version, and architecture.
- Example: `uname -a` (displays all system information).

```
goelashwin36@Ash: ~
File Edit View Search Terminal Help
goelashwin36@Ash:~$ uname -v
#31-Ubuntu SMP Tue Jul 17 15:39:52 UTC 2018
goelashwin36@Ash:~$
```

Figure 24: uname

14. du (Disk Usage):

- Shows the disk usage of files and directories.
- Example: `du -h` (displays human-readable file sizes).

```
[linuxopsys@machine01 ~]$ du -h /home/linuxopsys/projects/
0      /home/linuxopsys/projects/dir1/dir2-depth2
24M    /home/linuxopsys/projects/dir1
0      /home/linuxopsys/projects/dir2/dir2-depth2
1.0M   /home/linuxopsys/projects/dir2
50M    /home/linuxopsys/projects/
[linuxopsys@machine01 ~]$ du -h --max-depth=1 /home/linuxopsys/projects/
24M    /home/linuxopsys/projects/dir1
1.0M   /home/linuxopsys/projects/dir2
50M    /home/linuxopsys/projects/
[linuxopsys@machine01 ~]$
```

Figure 25: du

15. time (Measure Execution Time):

- Measures the execution time of a command.
- Example: `time ls`.

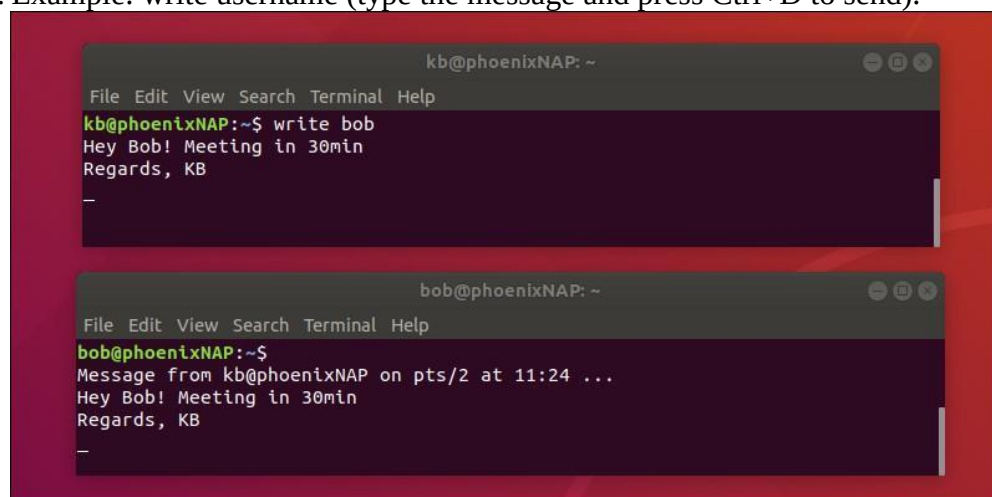
```
File Edit View Search Terminal Help
naman@root:~$ time sleep 3

real    0m3.003s
user    0m0.003s
sys     0m0.000s
naman@root:~$
```

Figure 26: time

16. write (Send a Message to a User):

- Sends a message to another logged-in user.
- Example: `write username` (type the message and press Ctrl+D to send).



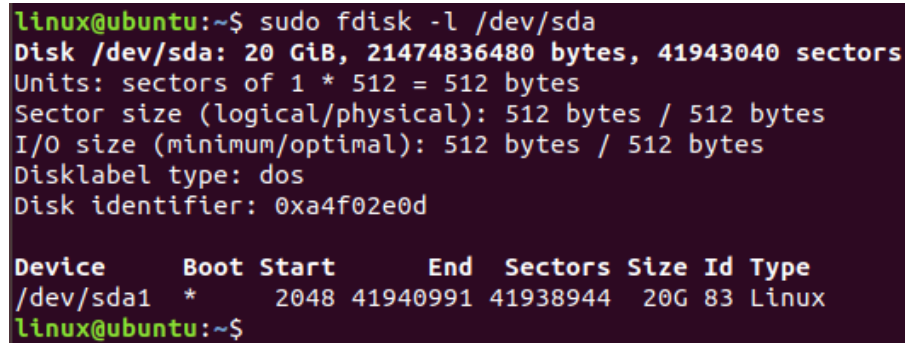
```
kb@phoenixNAP: ~
File Edit View Search Terminal Help
kb@phoenixNAP:~$ write bob
Hey Bob! Meeting in 30min
Regards, KB
_

bob@phoenixNAP: ~
File Edit View Search Terminal Help
bob@phoenixNAP:~$
Message from kb@phoenixNAP on pts/2 at 11:24 ...
Hey Bob! Meeting in 30min
Regards, KB
_
```

Figure 27: write

17. fdisk (Disk Partitioning Tool):

- a. Used to manage disk partitions.
- b. Example: `sudo fdisk -l` (lists all disk partitions).



```
linux@ubuntu:~$ sudo fdisk -l /dev/sda
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa4f02e0d

Device     Boot  Start      End  Sectors  Size Id Type
/dev/sda1  *    2048  41940991  41938944   20G 83 Linux
linux@ubuntu:~$
```

Figure 28: fdisk

MODEL VIVA QUESTIONS

1. How would you display the contents of a file named notes.txt, one page at a time?
2. Explain the difference between user, group, and others in the context of file permissions.
3. Explain the key functional difference between the cp (copy) and mv (move) commands.
4. What is the purpose of the touch command? Give two distinct examples of its use.
5. What is the purpose of the pwd command, and what command would you use to change your current directory to your home directory, regardless of your current location?

RESULT

Successfully executed and understood essential Linux commands such as ls, cd, pwd, mkdir, rm, cp, mv, and ps, gaining hands-on experience in managing files, directories, and processes in a Linux system.

EXP NO: 5**Familiarizing Computer Network Components****AIM**

To familiarize with various networking hardware components such as RJ45 connectors, UTP cables, fiber optic cables, switches, NICs, routers, Wireless Access Points (WAP), and modems, and understand their roles in establishing and maintaining network connectivity.

LEARNING OUTCOMES

Demonstrate the hardware, addressing, configuration, and design principles necessary to understand network components.

DESCRIPTION**1. RJ45 Connector**

- a. What it is: A standardized interface for connecting Ethernet cables to devices.
- b. Use: Primarily used to terminate UTP cables and connect them to devices like switches, routers, or computers. Appearance: An 8-pin modular connector.



Figure 29: RJ45

2. UTP Cable (Unshielded Twisted Pair)

- a. What it is: A type of cable with pairs of wires twisted together to reduce electro- magnetic interference.
- b. Use: Commonly used in Ethernet networks for connecting devices.
- c. Categories: CAT5e, CAT6, and CAT7 are popular types, with CAT6 and above supporting higher speeds.

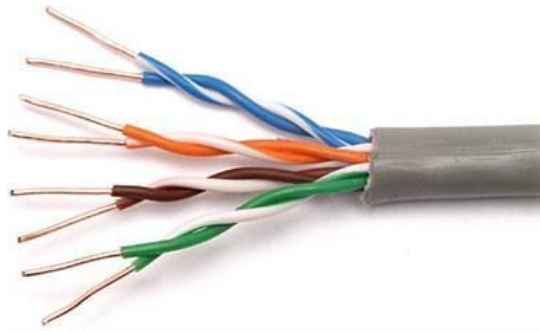


Figure 30: UTP Cable

3. Fibre (Optical Fiber Cable)

What it is: A high-speed data transmission cable using light signals instead of electrical signals.

- a. Use: Ideal for long-distance, high-bandwidth requirements like backbone connections in large networks.
- b. Types: Single-mode (long-distance) and Multi-mode (short-distance).



Figure 31: Fiber Optical Cable

4. Switch

- a. What it is: A device that connects multiple devices in a network and uses MAC addresses to forward data to the correct destination.
- b. Use: Facilitates communication between devices within the same local area network (LAN).
- c. Types: Managed and unmanaged switches.



Figure 32: Switch

5. NIC (Network Interface Card)

- a. What it is: A hardware component (internal or external) that connects a computer to a network.
- b. Use: Provides the physical interface for wired (Ethernet) or wireless (Wi-Fi) connectivity



figure 33: Wireless NIC Card

- c. Common Feature: Most modern NICs support high-speed data transfer.

6. Router

- a. What it is: A device that connects multiple networks and directs data packets between them.
- b. Use: Facilitates Internet access by connecting your local network to your ISP.
- c. Key Feature: Often includes a built-in firewall and additional security features.



Figure 34: Wifi Router

7. Wireless Access Point (WAP)

- a. What it is: A device that allows wireless devices to connect to a wired network using Wi-Fi. Use: Extends the range of a network or adds Wi-Fi capability.
- b. Difference from Router: A WAP alone doesn't manage IP addresses; it requires a router for full functionality.



Figure 35: Wireless Access Point

8. Modem

- a. What it is: A device that modulates and demodulates signals for Internet access.
- b. Use: Connects your home or office to your Internet Service Provider (ISP) via DSL, cable, or fiber.
- c. Key Feature: Often combined with a router in a single device for convenience.

MODEL VIVA QUESTIONS

1. Explain the key functional difference between a Switch and a Hub at the hardware level.
2. What is the primary function of a Router? How does it differ from a modem?
3. Describe the role of a Network Interface Card (NIC) and its relationship to the physical transmission medium.
4. Compare Fiber Optic cable and Twisted-Pair Copper cable in terms of speed, distance, and cost.
5. Differentiate between a MAC Address and an IP Address. Which one changes when a computer moves to a different network?

RESULT

Familiarized with various networking hardware components such as RJ45, UTP, fiber optic cables, switches, NICs, routers, WAPs, and modems, and understood their roles in network communication. Successfully established wired and wireless connections using these components to simulate a functional network setup.

EXP NO: 6**Familiarizing Basic Networking Commands****AIM**

To learn and practice basic networking commands in Linux/Windows to analyze, troubleshoot, and manage network connections.

LEARNING OUTCOMES

Demonstrate the basic networking commands in Linux/Windows to analyze, troubleshoot, and manage network connections.

DESCRIPTION

Networking commands are essential for diagnosing, configuring, and troubleshooting network connections in Unix/Linux environments. Below is an overview of some fundamental networking commands:

1. ifconfig (Interface Configuration):

- a. Displays or configures network interfaces on a system.
- b. Example: ifconfig (shows current network interface settings, IP address, and network statistics).
- c. To configure a network interface: `sudo ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up` (sets IP and netmask).

```

admincse@admincse-Lenovo-V330-15IKB:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:60:96:d2:2c txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp3s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.199.104.2 netmask 255.255.255.0 broadcast 10.199.104.255
    inet6 fe80::f69f:63ec:74b4:566f prefixlen 64 scopeid 0x20<link>
    ether 48:2a:e3:29:ad:ea txqueuelen 1000 (Ethernet)
    RX packets 24449 bytes 28425530 (28.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12499 bytes 1573356 (1.5 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1723 bytes 159300 (159.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1723 bytes 159300 (159.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp2s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 0e:61:c9:17:08:8a txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

2. ping (Send ICMP Echo Request):

- a. Tests the network connection between your machine and another system (by sending ICMP packets).
- b. Example: ping 8.8.8.8 (pings Google's DNS server to test network connectivity).
- c. Example: ping google.com (pings a domain name).

```

admincse@admincse-Lenovo-V330-15IKB:~$ ping 10.199.104.78
PING 10.199.104.78 (10.199.104.78) 56(84) bytes of data.
64 bytes from 10.199.104.78: icmp_seq=1 ttl=128 time=1.79 ms
64 bytes from 10.199.104.78: icmp_seq=2 ttl=128 time=0.839 ms
64 bytes from 10.199.104.78: icmp_seq=3 ttl=128 time=0.882 ms
64 bytes from 10.199.104.78: icmp_seq=4 ttl=128 time=0.914 ms
64 bytes from 10.199.104.78: icmp_seq=5 ttl=128 time=0.734 ms
64 bytes from 10.199.104.78: icmp_seq=6 ttl=128 time=0.687 ms
64 bytes from 10.199.104.78: icmp_seq=7 ttl=128 time=0.545 ms
64 bytes from 10.199.104.78: icmp_seq=8 ttl=128 time=0.963 ms
64 bytes from 10.199.104.78: icmp_seq=9 ttl=128 time=0.286 ms
64 bytes from 10.199.104.78: icmp_seq=10 ttl=128 time=0.916 ms
64 bytes from 10.199.104.78: icmp_seq=11 ttl=128 time=0.630 ms
^C
--- 10.199.104.78 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10131ms
rtt min/avg/max/mdev = 0.286/0.834/1.788/0.356 ms
admincse@admincse-Lenovo-V330-15IKB:~$

```

3. traceroute (Trace Network Route):

- a. Traces the path packets take from your machine to a destination, showing each hop along the way.
- b. Example: traceroute google.com (traces the route to Google's servers).

```

admincse@admincse-Lenovo-V330-15IKB:~$ traceroute google.com
traceroute to google.com (142.250.196.78), 30 hops max, 60 byte packets
 1 _gateway (10.199.104.1) 1.076 ms 0.992 ms 0.942 ms
 2 14.139.102.17 (14.139.102.17) 0.896 ms 0.885 ms 0.743 ms
 3 10.162.68.245 (10.162.68.245) 21.724 ms 21.883 ms 21.821 ms
 4 10.162.9.249 (10.162.9.249) 21.541 ms 21.701 ms 21.427 ms
 5 10.255.236.93 (10.255.236.93) 21.586 ms 21.620 ms 21.722 ms
 6 10.119.73.122 (10.119.73.122) 23.861 ms 24.175 ms 22.206 ms
 7 72.14.213.20 (72.14.213.20) 23.977 ms 23.817 ms 72.14.195.128 (72.14.195.128) 23.519 ms
 8 * * *
 9 216.239.43.172 (216.239.43.172) 24.265 ms 142.251.55.70 (142.251.55.70) 25.104 ms 142.251.55.232 (142.251.55.232) 24.138 ms
10 172.253.75.14 (172.253.75.14) 24.428 ms 142.250.236.157 (142.250.236.157) 24.021 ms 142.251.55.121 (142.251.55.121) 24.173 ms
11 naa03s46-in-f14.1e100.net (142.250.196.78) 23.982 ms 142.250.208.153 (142.250.208.153) 24.395 ms 142.251.50.59 (142.251.50.59) 22.050 ms

```

4. nslookup (Name Server Lookup):

- Queries DNS servers to obtain domain name or IP address information.
- Example: nslookup google.com (resolves the IP address of google.com).
- Example: nslookup 8.8.8.8 (finds the domain name for a given IP).

```

admincse@admincse-Lenovo-V330-15IKB:~$ nslookup google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.196.78
Name:   google.com
Address: 2404:6800:4007:819::200e

```

5. ssh (Secure Shell):

- Allows secure remote login and command execution on another system over a network.
- Example: ssh username@192.168.1.100 (remotely logs into a machine using SSH).
- Example: ssh -p 2222 username@192.168.1.100 (connects using a custom port).

```

admincse@admincse-Lenovo-V330-15IKB:~$ ssh username@192.168.1.10
ssh: connect to host 192.168.1.10 port 22: No route to host
admincse@admincse-Lenovo-V330-15IKB:~$ 

```

6. scp (Secure Copy):

- Used to securely copy files between a local and remote system or between two remote systems.
- Example: scp file.txt username@192.168.1.100:/path/to/destination/ (copies file.txt to a remote system).
- Example: scp username@192.168.1.100:/path/to/file.txt ./ (copies a file from a remote system to local).

7. telnet (Telnet Client):

- a. Connects to a remote system over the Telnet protocol (not encrypted, typically used for legacy systems).
- b. Example: telnet 192.168.1.100 23 (connects to a remote machine on port 23).
- c. Note: Telnet is not recommended for secure connections; SSH is preferred.

8. ftp (File Transfer Protocol):

- a. Transfers files between systems over a network using the FTP protocol.
- b. Example: ftp ftp.example.com (connects to an FTP server).
- c. Example: get file.txt (downloads a file from the FTP server).
- d. Example: put file.txt (uploads a file to the FTP server).

MODEL VIVA QUESTIONS

1. What is the primary Linux command used to view a computer's current IP address, subnet mask, and default gateway?
2. Which command is used to test network connection between your machine and another system?
3. Describe about ssh command.
4. Which command is used to securely copy files between a local and remote system or between two remote systems?

RESULT

Successfully executed and understood networking commands such as ping, ipconfig/ifconfig, traceroute, netstat, and nslookup, gaining hands-on experience in monitoring, diagnosing, and managing network connectivity.

EXP NO: 7**View network traffic using Wireshark/Packet tracer****AIM**

To understand how to capture, analyze, and interpret network traffic using tools like Wireshark and Packet Tracer for network monitoring and troubleshooting.

LEARNING OUTCOMES

To analyze, and interpret network traffic using tools like Wireshark and Packet Tracer for network monitoring and troubleshooting

DESCRIPTION**Cisco Packet Tracer**

Cisco Packet Tracer is a network simulation tool developed by Cisco Systems. It is primarily designed for learning and practicing network concepts, especially for students and professionals.

Key Features of Cisco Packet Tracer

- **Simulation of Network Devices:** Allows users to simulate and configure various network devices such as routers, switches, firewalls, and wireless access points.
- **Hands-On Practice:** Offers a virtual environment to practice network setup, configuration, and troubleshooting without requiring expensive hardware.
- **Interactive Learning:** Supports interactive activities and scenarios to help users understand networking concepts like IP addressing, routing, switching, and more.
- **Real-Time Network Behavior:** Simulates real-time data flow between devices to demonstrate how networks operate.
- **Cross-Platform Availability:** Works on Windows, macOS, Linux.

Benefits of Cisco Packet Tracer

- **Free for Students:** Provided for free through Cisco Networking Academy.
- **Risk-Free Experimentation:** Allows users to test configurations without affecting real-world systems.

- **Visual Learning:** Provides a graphical interface to visualize network designs and operations.
- **Scalable:** Supports small to large-scale network simulations. Exploring the GUI of the Cisco Packet Tracer

Once you open the Cisco Packet Tracer following GUI screen appear.

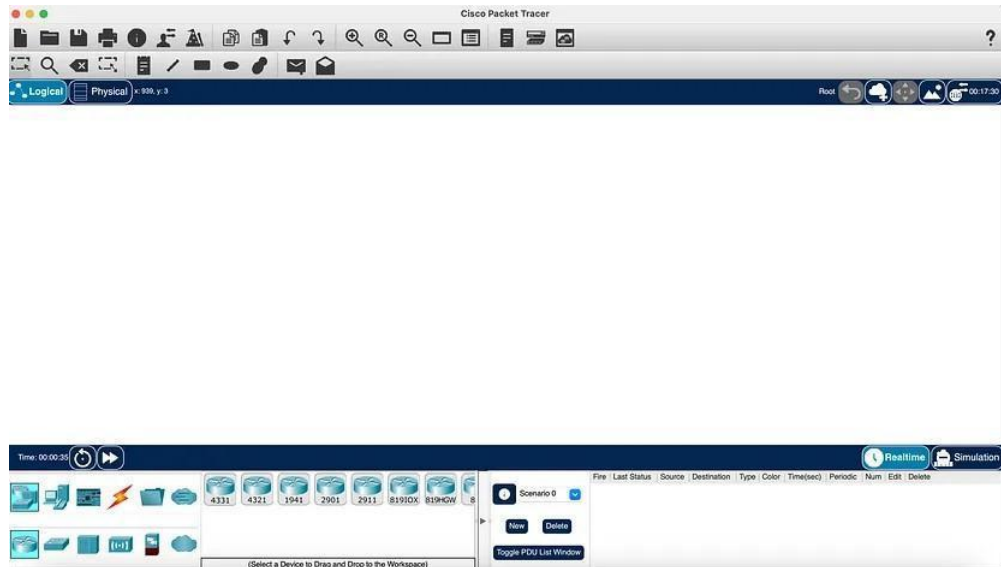


Figure 36: Interface of Cisco

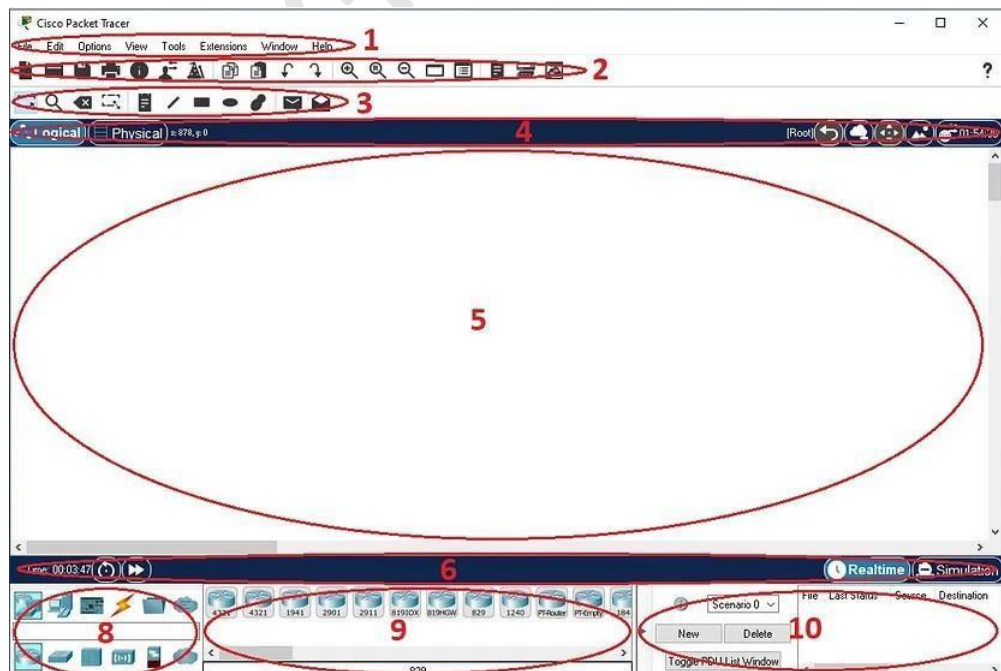


Figure 37: Interface Overview

This initial interface contains ten components:

1. **Menu Bar:** This bar provides the File, Edit, Options, View, Tools, Extensions,

Win- dow, and Help menus. You will find basic commands such as Open, Save, Save as Pkz, Print, and Setting and Preferences in these menus.

2. **Main tool Bar:** This bar provides shortcut icons to the most commonly used menu commands
3. **Common Tool Bar:** This bar provides access to these commonly used workspace tools: Select, Inspect, Delete, Resize Shape, Place Notes, Drawing Palette, Add Simple PUD, and Add Complex PDU.
4. **Logical/Physical Workspace and Navigation Bar:** You can switch or toggle between Physical Workspace and Logical Workspace with the tabs on this bar. In Logical Workspace, this bar allows you to go back to a previous level in the cluster, create a New Cluster, Move object, and set a Tiled background and viewpoint, etc. In a physical Workspace, this bar allows you to navigate through physical locations, create a new city, create a new building, create a new closet, move an object, apply a Grid to the background, etc.
5. **Workspace:** This area is where you will create your network, watch simulations, and view many kinds of information and statistics.
6. **Realtime/Simulation Bar:** You can toggle between Realtime Mode and Simulation Mode with the tabs on this bar. This bar also provides buttons to Power Cycle Devices and Fast Forward Time as well as the Play control buttons and the event list toggle button in Simulation Mode.
7. **Network Component Box:** This box is where you choose devices and connections to put into the workspace. It contains the Device-Type Selection Box and the Device- Specific Selection Box.
8. **Device-Type Selection Box:** This box contains the type of devices and connections available in the packet tracer. This box will change depending on which type of device you select.
9. **Device-Specific Selection Box:** This box is where you choose specifically which device you want to add to your network and which connection to make.
10. **User Created Packet Window:** This window manages the packets you add to the network during simulation scenarios.

MODEL VIVA QUESTIONS

- 1) Explain the functional difference between generating controlled traffic in **Packet Tracer** versus capturing real-time traffic using **Wireshark**.
- 2) If your computer has multiple network interfaces (e.g., Wi-Fi and Ethernet), how do you select the correct interface in Wireshark to ensure you capture the intended traffic?

RESULT

Successfully captured and analyzed network traffic using Wireshark/Packet Tracer, gaining insights into network protocols, packet structures, and identifying potential issues.

EXP NO: 8

Familiarizing the steps how to configure and establishing a network connection

AIM

To familiarize with the steps involved in configuring and establishing a network connection, including setting up devices, configuring IP addresses, and verifying connectivity.

LEARNING OUTCOMES

Students will learn to configure network settings and establish a functional connection between computer systems, verifying connectivity using basic networking commands.

DESCRIPTION

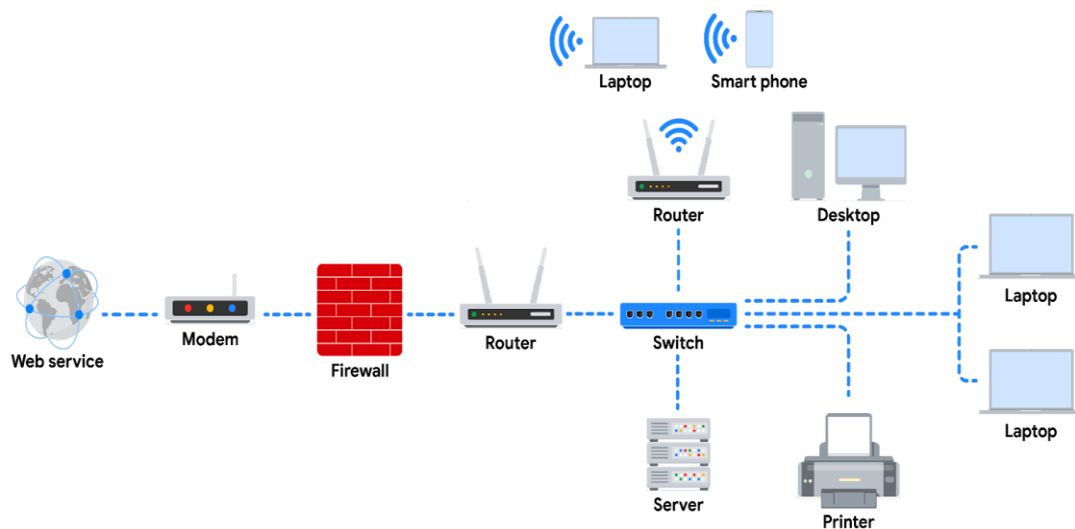


figure 38: Network

Configure Each Device

1. Configure the Modem

- Connect to ISP:** Use the provided credentials to configure the modem to establish the internet connection.
- Bridge Mode (Optional):** If you're using an external router, set the modem to bridge mode to avoid double NAT.

2. Configure the Router

- Connect the Modem to the Router:** Use an Ethernet cable to connect the modem's LAN port to the router's WAN port.
- Access the Router's Web Interface:**

- i. Connect a PC to the router (via Ethernet or Wi-Fi).
 - ii. Open a browser and enter the router's IP address (usually 192.168.0.1 or 192.168.1.1).
 - iii. Login with admin credentials.
- c. **Set Up Internet Connection:**
- i. Use Dynamic Host Configuration Protocol (DHCP) if the ISP assigns an IP automatically.
 - ii. Use static IP or PPPoE if required by the ISP.
- d. **Configure LAN Settings:**
- i. Set the router's LAN IP range (e.g., 192.168.1.1/24).

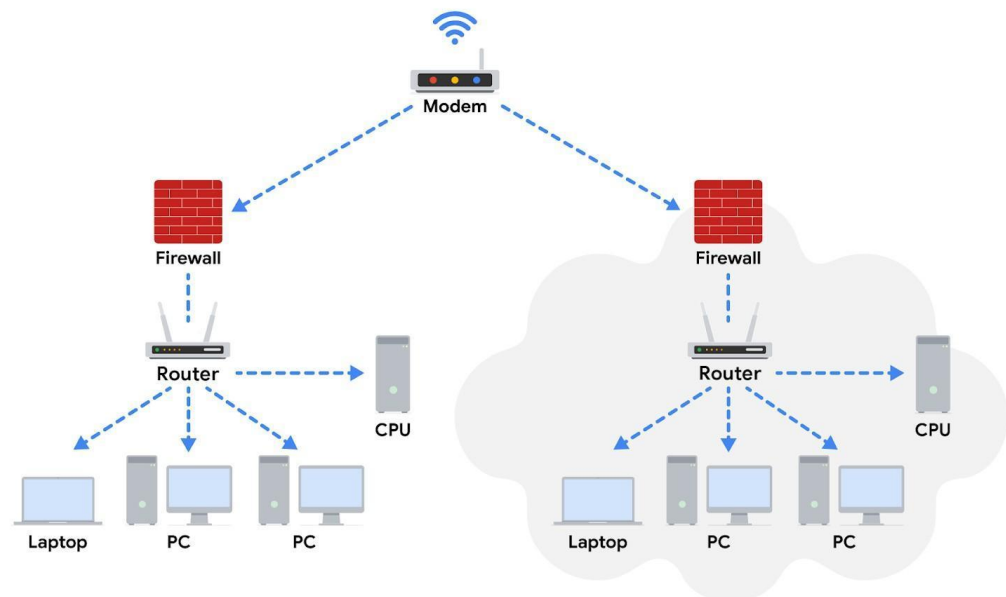


Figure 39: Caption

- ii. Enable DHCP to automatically assign IPs to connected devices.
- e. **Secure the Network:**
- i. Change default admin username/password.
 - ii. Set up a firewall.
 - iii. Enable WPA3/WPA2 encryption for Wi-Fi.

3. Configure the Switch

- a. **Connect the Switch to the Router:** Use an Ethernet cable to connect one of the router's LAN ports to the switch.
- b. **Configure VLANs (Optional):** If your switch is managed, create VLANs to segment the network for better performance and security.

- c. **Connect End Devices:** Plug devices (PCs, printers, etc.) into the switch's Ethernet ports.

4. Configure the Wireless Access Point

- a. **Connect WAP to the Router or Switch:** Use an Ethernet cable to connect to the LAN port.
- b. **Access the WAP's Web Interface:**
 - i. Connect to the WAP using its default Wi-Fi or Ethernet.
 - ii. Open a browser and enter the WAP's IP address (check documentation).
- c. **Set Up Wi-Fi:**
 - i. Configure the SSID (Wi-Fi name) and password.
 - ii. Set the security mode (WPA3/WPA2 preferred).
 - iii. Enable dual-band (2.4 GHz and 5 GHz) if supported.
- d. **Place the WAP Strategically:**
 - i. Ensure optimal coverage by placing the WAP in a central location, free from physical obstructions.

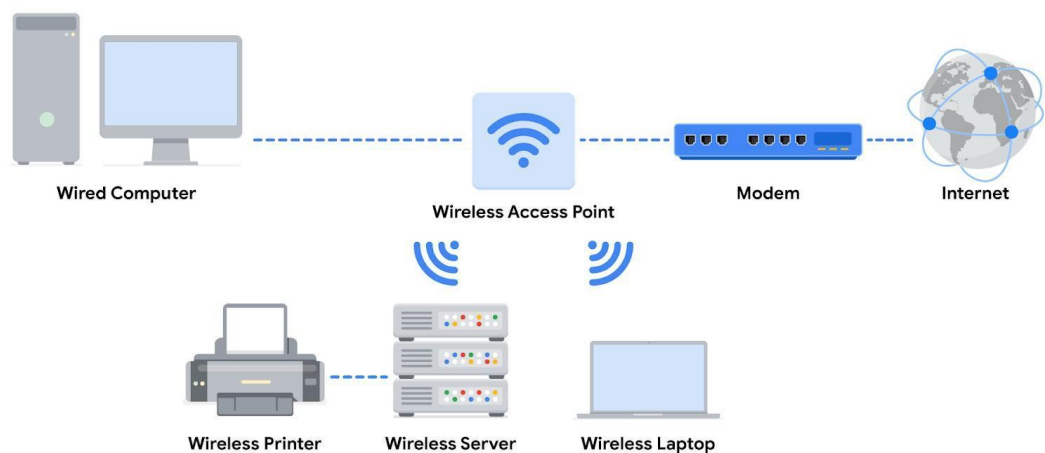


Figure 40: WAP

5. Configure the PCs

- a. **Set Network Settings:**
 - i. Use DHCP (default) to get IPs from the router automatically.
 - ii. Configure static IPs if required.

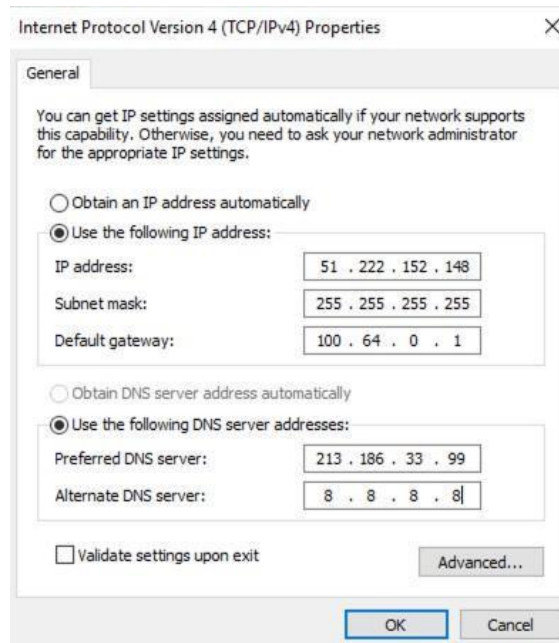


Figure 41: Configuration on Windows

b. Test Connectivity:

- i. Ping the router's IP address to verify LAN connectivity.
- ii. Ping a public IP or website (e.g., 8.8.8.8) to check internet access.

MODEL VIVA QUESTIONS

1. What is network configuration?
2. Why is network configuration important?
3. What is an IP address?
4. What is the difference between IPv4 and IPv6?
5. What is a subnet mask, and what is its purpose?
6. What is a default gateway?

RESULT

Successfully configured and established a network connection, ensuring proper communication between devices and verifying connectivity using basic network troubleshooting tools.

EXP NO: 9**Shell programming in Linux (bash)****AIM**

To understand and implement basic shell programming in Linux using Bash, including creating and executing scripts to automate tasks.

LEARNING OUTCOMES

Understand shell programming in Linux using Bash, including creating and executing scripts to automate tasks.

SOFTWARE REQUIRED

Linux Operating System : Bash/Terminal

DESCRIPTION

Shell programming in Linux refers to writing scripts using a shell (command-line interpreter) to automate tasks. A shell script is a file containing a series of commands that the shell executes in sequence.

The most commonly used shell in Linux is **Bash** (Bourne Again Shell), but there are others like Zsh, Ksh, and Fish. Shell scripts are used to automate repetitive tasks, manage system configurations, and simplify complex operations.

Advantages of Shell Programming

- Automation: Simplifies repetitive tasks (e.g., backups, file management).
- Efficiency: Executes complex tasks quickly with fewer commands.
- Portability: Can run across different Linux distributions with minimal modifications.

Basic Structure of a Shell Script

- Shebang Line: Indicates the script interpreter (#!/bin/bash).
- Commands: Series of Linux commands and logic.

bash

 Copy code

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

Execution:

- Save the script as filename.sh.
- Make it executable: `chmod +x filename.sh`.
- Run: `./filename.sh`.

CODE**Program 1: A Simple Greeting Script**

This script greets the user with their name.

bash

 Copy code

```
#!/bin/bash

# Ask for the user's name
echo "Enter your name:"
read name

# Greet the user
echo "Hello, $name! Welcome to shell scripting."
```

Program 2: Adding Two Numbers

This is simple Bash script to add two numbers.

bash

 Copy code

```
#!/bin/bash

# Prompt the user to enter the first number
echo "Enter the first number:"
read num1

# Prompt the user to enter the second number
echo "Enter the second number:"
read num2

# Add the two numbers
sum=$((num1 + num2))

# Display the result
echo "The sum of $num1 and $num2 is $sum."
```

PROCEDURE

1. Open a text editor(nano or vim)
2. Create the script file (eg. nano greeting.sh)
3. Add the shell script code
4. Save and exit(If using nano, press Ctrl + O to save and Ctrl + X to exit.)
5. Make the script executable (eg. chmod +x greeting.sh)
6. Run the program (./greeting.sh)
7. Provide input
8. View output

EXP NO: 9.A**Basic arithmetic operations and input/output handling in shell programming.****AIM**

To write a shell script that reads two numbers from the user, performs addition, and displays the result, demonstrating basic arithmetic operations and input/output handling in shell programming.

ALGORITHM

- Start
- Display a message asking the user to enter the first number
- Read the first number and store it in variable a
- Display a message asking the user to enter the second number
- Read the second number and store it in variable b
- Add the two numbers and store the result in variable sum
- Display the result (sum) to the user
- End

PROGRAM

```
Echo "Enter first number:"  
  
read a  
  
echo "Enter second number:"  
  
read b  
  
sum=$((a + b))  
  
echo "The sum of $a and $b is: $sum"
```

OUTPUT

```
./add_numbers.sh  
  
Enter first number:5  
  
Enter second number:7  
  
The sum of 5 and 7 is: 12
```

EXP NO: 9.B**Shell Script to Check Whether a Number is Positive, Negative, or Zero****AIM**

To write a shell script that reads a number from the user and uses the if statement to determine whether the number is positive, negative, or zero, and display the result.

ALGORITHM**Start**

1. Display a message asking the user to **enter a number**
2. Read the number and store it in variable num
3. Check if num is greater than 0
 If yes, display "num is positive"
4. Else, check if num is less than 0
 If yes, display "num is negative"
5. Else, display "num is zero"
6. End

PROGRAM

```
echo "Enter a number:"  
  
read num  
  
if [ $num -gt 0 ]  
then  
    echo "The number $num is positive."  
elif [ $num -lt 0 ]  
then  
    echo "The number $num is negative."  
else
```

```
echo "The number is zero."
```

```
fi
```

OUTPUT

```
./check_number.sh  
Enter a number:  
8  
The number 8 is positive.
```

```
./check_number.sh  
Enter a number:  
-3  
The number -3 is negative.
```

EXP NO: 9.C**MULTIPLICATION TABLE****AIM**

To write a shell script that reads a number from the user and displays its multiplication table from 1 to 10 using a for loop.

ALGORITHM

- Start
- Display a message asking the user to **enter a number**
- Read the number and store it in variable num
- Display a message: "Multiplication table of num:"
- Use a for loop from 1 to 10:
- Multiply num by the loop variable i
- Display the result as "num x i = result"
- End

PROGRAM

```
echo "Enter a number:"
read num

echo "Multiplication table of $num:"

for i in {1..10}
do
    result=$((num * i))
    echo "$num x $i = $result"
Done
```

OUTPUT

```
./multiplication_table.sh
Enter a number:
7
Multiplication table of 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
```

7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

MODEL VIVA QUESTIONS

1. Store the output of the command “hostname” in a variable. Display “This script is running on .” where “ ” is the output of the “hostname” command.
2. Write a shell script that displays “man”, “bear”, “pig”, “dog”, “cat”, and “sheep” on the screen with each appearing on a separate line.
3. Write a shell script that consists of a function that displays the number of files in the present working directory. Name this function “file count” and call it in your script. If you use variable in your function, remember to make it a local variable.
4. Write a shell script to convert string lower to upper and upper to lower.
5. Implement “while loop” using shell scripting.
6. Implement “for loop” using shell scripting.

RESULT

Successfully created and executed shell scripts in Linux to automate tasks such as arithmetic operations, file handling, and displaying system information, demonstrating the fundamentals of Bash scripting.

EXP NO :10**Form Validation using JavaScript****AIM**

To use Javascript to validate forms.

LEARNING OUTCOMES

Students will learn to validate user inputs on web forms using JavaScript, display dynamic error messages, and prevent incorrect submissions. They will also gain practical skills in using DOM manipulation, regular expressions, and integrating HTML, CSS, and JavaScript for interactive forms.

SOFTWARE REQUIRED

Web Browser
code Editor

DESCRIPTION

JavaScript is a high-level, interpreted programming language used to make web pages interactive and dynamic. It allows developers to manipulate the content, structure, and style of web pages, respond to user actions, and perform calculations directly in the browser. JavaScript works alongside HTML and CSS, and is widely used for tasks like form validation, animations, interactive buttons, and creating web applications. It can run on both the client side (in the browser) and server side (using Node.js).

ALGORITHM

- Start
- When the user submits the form, call the validateForm() function.
- Inside validateForm():
 - a. Call clearErrors() to remove any previous error messages.
 - b. Read the input values for name, email, and password.
 - c. Initialize a variable isValid = true to track validation status.
- Validate Name:
 - If the name is empty, display "Name is required" using showError() and set isValid = false.
 - Else if the name length is less than 3 characters, display "Name must be at least 3 characters long" and set isValid = false.

- Validate Email:
- If the email is empty, display "Email is required" and set isValid = false.
- Else if the email does not match the email format regex, display "Enter a valid email address" and set isValid = false.
- Validate Password:
- If the password is empty, display "Password is required" and set isValid = false.
- Else if the password length is less than 6 characters, display "Password must be at least 6 characters long" and set isValid = false.
- Return the value of isValid to determine if the form should be submitted.
- Helper Functions:
- showError(elementId, message): Display the error message in the corresponding <small> element.
- clearErrors(): Clear all previous error messages from the form.
- End

PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Validation</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="form-container">
    <form id="registrationForm" onsubmit="return validateForm()">
      <h2>Registration Form</h2>

      <!-- Name Field -->
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" placeholder="Enter your name">
      <small class="error" id="nameError"></small>
```

```

        <!-- Email Field -->
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" placeholder="Enter your email">
        <small class="error" id="emailError"></small>

        <!-- Password Field -->
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" placeholder="Enter your
        <small class="error" id="passwordError"></small>

        <!-- Submit Button -->
        <button type="submit">Register</button>
    </form>
</div>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

    body {
        font-family: Arial, sans-serif;
        margin: 0;
        padding: 0;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
        background-color: #f7f7f7;
    }

    .form-container {
        background: white; padding: 20px; border-radius: 10px;
        box-shadow: 0 4px 10px
            rgba(0, 0, 0, 0.2);
        width: 300px;
    }

    form h2 {
        text-align: center; margin-bottom: 20px;
    }

    label {
        font-weight: bold; display: block; margin: 10px 0 5px;
    }

    input {

```

```
width: 100%; padding: 10px; margin-bottom: 10px;
border: 1px solid
#ccc; border-
radius: 5px;
}

button {
width: 100%; padding: 10px;
background-color: #007BFF; color: white;
border: none; border-radius: 5px; cursor: pointer;
}

button:hover {
background-color: #0056b3;
}

.error {
color: red;
font-size:
0.9em; margin-
top: -8px;
margin-bottom:
10px;
}
```

script.js

```
function validateForm() {
// Clear previous
errors clearErrors();

// Get form values
const name = document.getElementById("name").value.trim();
const email = document.getElementById("email").value.trim();
const password = document.getElementById("password").value.trim(); let

isValid = true;

// Validate
Name if
(name ===
"") {
showError("nameError", "Name is required.");
isValid = false;
} else if (name.length < 3) {
showError("nameError", "Name must be at least 3 characters long.");
isValid = false;
}

// Validate Email
const emailRegex =
/^[^\s@]+@[^\s@]+\.[^\s@]+$/; if (email === "")
```

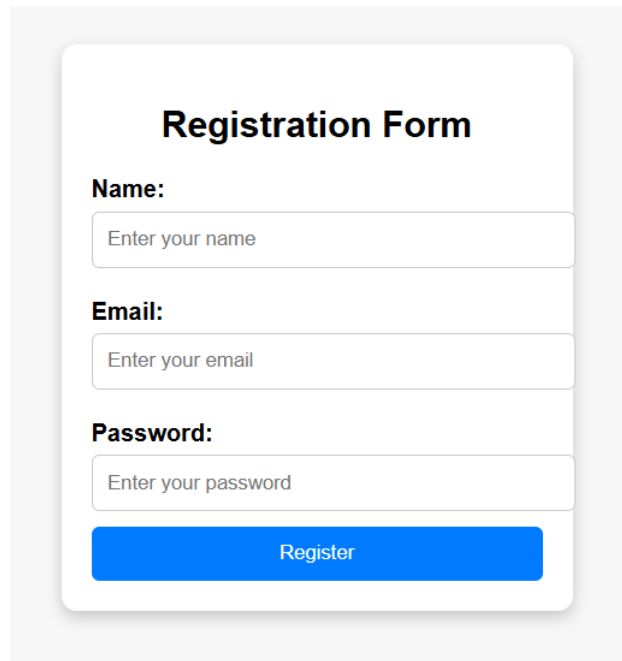
```
{
  showError("emailError", "Email is required.");
  isValid = false;
} else if (!emailRegex.test(email)) { showError("emailError",
  "Enter a valid email address."); isValid = false;
}

// Validate
// Password if
// (password ===
// "") {
  showError("passwordError", "Password is required.");
  isValid = false;
} else if (password.length < 6) {
  showError("passwordError", "Password must be at least 6 characters long."); isValid = false;
}

// Return validation status
return isValid;
}

// Helper function to show error messages
function showError(elementId, message)
{
  const errorElement = document.getElementById(elementId);
  errorElement.innerText = message;
}

// Helper function to clear all error messages function
clearErrors() {
  const errorElements = document.querySelectorAll(".error"); errorElements.forEach((element) =>
  (element.innerText = ""));
}
```



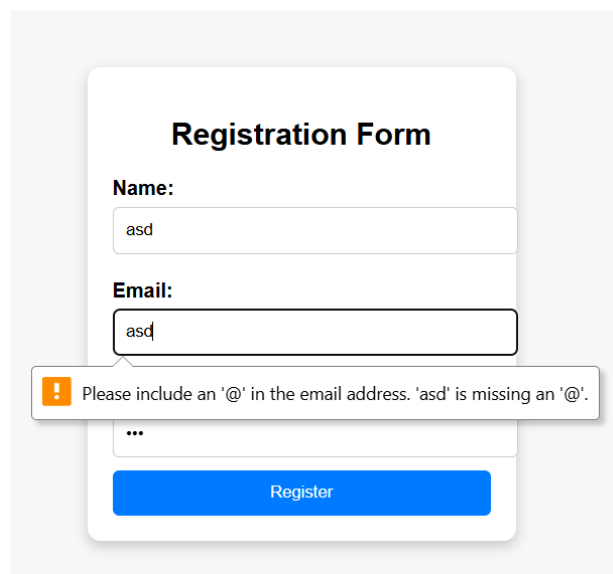
Registration Form

Name:

Email:

Password:

Register



Registration Form

Name:

Email:

⚠ Please include an '@' in the email address. 'asd' is missing an '@'.

...

Register

PROCEDURE

- Open a code editor
- Create the HTML file
 - Click File → New File

- Type the HTML code exactly as given:
 - Form structure
 - Name, Email, Password fields
 - `<small>` elements for error messages
 - `<script src="script.js"></script>` at the bottom
- Save the file as index.html (or form.html).
- Create the CSS file
 - a. Create another new file
 - b. Type the CSS styling that formats your form, buttons, and error text
 - c. Save this file as styles.css
- Create the JavaScript file
 - Create a third new file
 - Type the JavaScript validation code:
 - `validateForm()`
 - `showError()`
 - `clearErrors()`
 - Save this file as script.js
- Ensure all three files are in the same folder
- Open the HTML file in a browser
- Browser loads the form interface
- User enters information in the form clicks “REGISTER” Button

MODEL VIVA QUESTIONS

1. What is the difference between JavaScript and HTML?
2. What is JavaScript and how is it used in web development?
3. What is the purpose of the `document.getElementById()` method?
4. What are functions in JavaScript?

RESULT

Successfully implemented form validation using JavaScript to ensure that user inputs, such as name, email, and password, meet specified criteria before submission, enhancing data accuracy and user experience.

EXP NO :11**Image Slider using HTML, CSS and JavaScript****AIM**

Create an image slider using HTML, CSS, and JavaScript. Allow users to navigate between images using previous and next buttons.

LEARNING OUTCOMES

The student learns how to create an interactive image slider using HTML, CSS, and JavaScript

SOFTWARE REQUIRED

Web Browser
code editor

ALGORITHM

1. Start
2. Set currentSlide = 0.
3. Define function showSlide(index).
4. Get all .slide images.
5. Get the .slider container.
6. Calculate total number of slides.
7. If $\text{index} \geq \text{number of slides}$, reset currentSlide = 0.
8. If $\text{index} < 0$, set currentSlide = last slide index.
9. Calculate offset using $-\text{currentSlide} * 100\%$.
10. Move slider using CSS transform.
11. Define function nextSlide() to increment slide index and call showSlide().
12. Define function prevSlide() to decrement slide index and call showSlide().
13. Call showSlide(0) to display the first image.
14. End.

PROGRAM

index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Image Slider</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="slider-container">
    <div class="slider">
      
      
      
    </div>
    <button class="prev" onclick="prevSlide()">&#10094;</button>
    <button class="next" onclick="nextSlide()">&#10095;</button>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  display:
  flex;
  justify-content:
  center; align-items:
  center; height:
  100vh;
  background-color: #f4f4f4;
}

.slider-container {
  position:
  relative; width:
  80%;
  max-width:
  600px;
  overflow:
  hidden;
  border: 2px
  solid #ddd;
  border-
  radius:
  10px;
```

```
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
    }

.slider {
    display: flex;
    transition: transform 0.5s ease-in-out;
}

.slide {
    width:
    100%;
    height:
    auto;
    display:
    block;
}

button {
    position:
    absolute; top:
    50%;
    transform: translateY(-50%);
    background-color: rgba(0, 0, 0, 0.5);
    color: white;
    border: none;
    border-radius:
    50%; width:
    40px; height:
    40px; cursor:
    pointer;
    z-index:
    10; font-
    size:
    18px;
    outline:
    none;
}

button:hover {
    background-color: rgba(0, 0, 0, 0.7);
}

.prev {
    left: 10px;
}

.next {
    right: 10px;
}
```

script.js

```
let currentSlide = 0;

function showSlide(index) {
  const slides = document.querySelectorAll(".slide");
  const slider = document.querySelector(".slider");
  const totalSlides = slides.length; // Wrap around logic for
  navigation if (index >= totalSlides) {
    currentSlide = 0;
  } else if (index < 0) { currentSlide
    = totalSlides - 1;
  } else {
    currentSlide = index;
  }

  // Update the transform property to slide images
  const offset = -currentSlide * 100;
  slider.style.transform = `translateX(${offset}%)`;
}

function nextSlide() {
  showSlide(currentSlide + 1);
}

function prevSlide() {
  showSlide(currentSlide - 1);
}

// Initialize the first slide
showSlide(currentSlide);
```

OUTPUT



MODEL VIVA QUESTIONS

1. Which HTML tag did you use to display the images in the slider?
2. How does JavaScript control the automatic sliding of images?
3. What is setInterval() and how is it used in the image slider?

RESULT

Successfully created an interactive image slider using HTML, CSS, and JavaScript, allowing users to navigate between images using "Previous" and "Next" buttons, ensuring smooth transitions and a responsive design.

EXP NO :12**Familiarisation of LaTeX****AIM**

To familiarize with the basics of LaTeX, including document structure, text formatting, and compiling a simple document.

LEARNING OUTCOMES

Students will learn to create, format, and compile documents using LaTeX, including handling sections, text formatting, equations, tables, and figures. They will gain skills in writing professional-quality documents with structured formatting, automated numbering, and mathematical typesetting.

SOFTWARE REQUIRED

LaTeX editor/IDE: **Overleaf (online)**

DESCRIPTION

LaTeX (pronounced Lay-tech or Lah-tech) is a typesetting system commonly used for scientific, technical, and mathematical documents. It produces high-quality, professional-looking documents, especially useful for academic papers, theses, reports, and research articles.

LaTeX is not a word processor like Microsoft Word. Instead, it uses plain text and markup commands to format and typeset documents.

Why Use LaTeX?

LaTeX is ideal because:

1. **High-quality typesetting:** Especially for mathematical symbols and complex documents.
2. **Automatic numbering:** Handles figures, tables, references, and numbering automatically.
3. **Customization:** You can tweak almost every part of the document's design.
4. **Cross-platform:** Works on Windows, macOS, and Linux.

LaTeX uses markup commands in plain text to describe how a document should look. These commands tell LaTeX how to format the text, equations, tables, figures, and other

elements.

1. Document Structure

These commands set up the structure of any LaTeX document:

`\documentclass{article}`: Defines the type of document (article, book, report, etc.).

`\begin{document}`: Marks the start of the main document content.

`\end{document}`: Marks the end of the document.

`\title{Your Title}`: Sets the document's title.

`\author{Your Name}`: Specifies the author.

`\date{}`: Adds the date (e.g., `\date{\today}` for today's date).

`\maketitle`: Generates the title page with title, author, and date.

Structure:

```
\documentclass{article} % Select the document type
```

```
\usepackage[utf8]{inputenc} % Package for UTF-8 support
```

```
\begin{document} % Start the document content
```

```
\title{My First LaTeX Document}
```

```
\author{Your Name}
```

```
\date{\today}
```

```
\maketitle % Generates the title page
```

Hello, world! This is my first document in LaTeX.

```
\section{Introduction}
```

This is a basic introduction section.

```
\end{document} % End the document content
```

2. Sections and Subsections

Commands to divide your document into sections:

`\section{}`: Creates a top-level section.

`\subsection{}`: Creates a subsection.

`\subsubsection{}`: Creates a sub-subsection.

`\paragraph{}`: Creates a small paragraph-like heading.

3. Text Formatting

These commands are used for common text formatting:

`\textbf{}`: Makes text bold.

`\textit{}`: Makes text italic.
`\underline{}`: Underlines text.
`\texttt{}`: Displays text in monospace font (typewriter-like).
`\emph{}`: Emphasizes text (typically italicized).

4. Mathematical Expressions

LaTeX excels at handling math with these basic commands:

`$...$`: Inline math mode. Example: $x^2 + y^2$.
`\[...\]`: Displayed math mode.
`\frac{a}{b}`: Creates fractions. Example: $\frac{1}{2}$.
`\sum`: Summation symbol.
`\int`: Integral symbol.
`\sqrt{}`: Square root. Example: $\sqrt{4}$.

5. Lists

Commands for creating lists:

Unordered List (Bulleted)

```
\begin{itemize}
  \item Item 1
  \item Item 2
  \item Item 3\end{itemize}
```

Ordered List (Numbered)

```
\begin{enumerate}
  \item Item 1
  \item Item 2
  \item Item 3
\end{enumerate}
```

6. Including Images

To include images with the `graphicx` package:

Add the package:

```
\usepackage{graphi
```

```
cx} Insert an image:
```

```
\begin{figure}[h]
  \centering
  \includegraphics[width=0.5\textwidth]{image.png}
```

```

\caption{My image.}
\label{fig:myimage}
\end{figure}

```

7. Tables

Commands to create tables:

```

\begin{tabular}{|c|c|c|}
\hline
A1 & B1 & C1 \\
\hline
A2 & B2 & C2 \\
\hline
\end{tabular}

```

8. Citations

For managing references, use \cite and define references: Define the bibliography

Add to a .bib

file: bibtex

```

@article{example,author =
{John Doe},title = {Example
Article},

```

```

journal={Sample Journal}, year = {2020}
}

```

Use in document:

```
\bibliographystyle{plain}
```

```
\bibliography{yourfi
```

```
le}
```

And cite:

```
\cite{example}
```

9. Tables of Contents

Insert a table of contents with:

```
\tableofcontents
```

10. Page Layout

Use the geometry package to change margins:

```
\usepackage[margin=1in]{geometry}
```

11. Creating Hyperlinks

Add hyperlinks using the hyperref package:

```
\usepackage{hyperref} Then use:  
\href{https://www.example.com}{Click here to visit example.com}
```

12. Environments

LaTeX uses environments for specific tasks:

itemize: Unordered lists.
enumerate: Ordered lists.
figure: Insert images and figures.
table: Insert tables.
align: For aligning mathematical equations.

13. Page Breaks

You can force a new page using:

\newpage: Ends the current page.
\pagebreak: Suggests a page break.

14. Custom Commands

You can create custom shortcuts with \newcommand.

Example:

```
\newcommand{\myname}{John Doe}
```

Usage:

Hello, my name is \myname.

15. Comments

Comments in LaTeX are denoted with %. LaTeX ignores these during compilation.

```
% This is a comment
```

This text will appear in the document.

LaTeX Tools

Here are a few tools that are helpful for LaTeX users:

- a. **Overleaf** : An online LaTeX editor. No installation is required. Overleaf website
- b. **TeX Live**: A LaTeX distribution that includes all LaTeX tools and packages.
- c. **MikTeX**: Another LaTeX distribution, mostly used for Windows users.

d. **Editors:** TeXworks, TeXShop, or LyX as offline LaTeX editors.

Common LaTeX Packages

Packages add additional features. Some common ones:

- e. **graphicx:** For including images.
- f. **amsmath:** For advanced math formatting.
- g. **geometry:** For customizing margins.
- h. **hyperref :** Adds hyperlinks.
- i. **biblatex:** For advanced bibliography management.

MODEL VIVA QUESTIONS

- 1) What is the basic structure of a LaTeX document?
- 2) What is the purpose of `\documentclass{}`?
- 3) What are the common file extensions used in LaTeX?
- 4) How do you apply bold, italics, and underline in LaTeX?
- 5) How can you insert lists (ordered and unordered) in LaTeX?

RESULT

Successfully created and compiled a basic LaTeX document, demonstrating understanding of its structure and basic commands.

EXP NO :13**Familiarisation of Development Environments****AIM**

To familiarize with popular development environments such as Visual Studio Code, Sublime Text, and Atom, focusing on their features, interface, and basic usage for programming and text editing.

LEARNING OUTCOMES

Students will learn to efficiently use modern development environments like VS Code, Sublime Text, and Atom for writing, editing, and organizing code. They will gain skills in utilizing features such as syntax highlighting, auto-completion, debugging, integrated terminals, and extensions to enhance productivity and manage projects effectively

DESCRIPTION

A development environment is the workspace where programmers create, test, and refine their code. Think of it as a virtual toolbox for building software. It provides a structured setup like:

- j. **Write Code:** Use editors to create scripts and programs.
- k. **Run Code:** Execute the programs to see the output or identify errors.
- l. **Debug Code:** Find and fix issues in your code.

Some of popular IDE's are :**1. Visual Studio Code (VS Code)**

Visual Studio Code (VS Code) is a free, open-source, and highly versatile code editor developed by Microsoft. It is widely recognized as one of the best tools for beginners stepping into the world of programming, offering a combination of simplicity and powerful features. Let's delve deeper into why VS Code is an excellent choice and explore its features and benefits in detail.

Why Choose Visual Studio Code?

VS Code stands out among other editors due to its lightweight design, ease of use, and extensive functionality. It supports multiple programming languages right out of the box, making it a one-stop solution for learners exploring various coding paths like web development, data science, and app development.

Key Features

- a. **IntelliSense: Smarter Coding Made Easy** - IntelliSense is a standout feature of VS Code that offers intelligent code suggestions, autocomplete, and syntax highlighting. This makes coding more efficient by reducing the likelihood of errors and helping beginners understand the structure and syntax of programming languages.
- b. **Built-in Terminal** - The integrated terminal allows you to run commands, test scripts, or execute programs directly within the editor. This eliminates the need to switch between applications, streamlining your workflow and saving time.
- c. **Extensions Marketplace** - The Extensions Marketplace in VS Code is a treasure trove of tools that can enhance your coding experience. With just a few clicks, you can add support for specific programming languages (like Python, JavaScript, or C++), frameworks (like React or Angular), or utilities (like linters and debuggers). Beginners can explore extensions like Prettier (for code formatting) and Live Server (for real-time web development).
- d. **Version Control with Git Integration** - Git integration is built directly into VS Code, enabling you to track changes, commit updates, and collaborate with others through platforms like GitHub. For beginners, this provides an excellent opportunity to learn version control—a critical skill for developers.
- e. **Debugging Made Simple** - VS Code includes debugging tools that help you identify and fix errors in your code. For beginners, this feature provides a clear and interactive way to understand program behavior, making debugging less intimidating.
- f. **Cross-Platform Support** - Whether you're using Windows, macOS, or Linux, VS Code is available for all major operating systems, ensuring accessibility and consistency across devices.
- g. **Customization and Themes** - VS Code allows you to personalize the editor to your liking. You can choose from a variety of themes, fonts, and layout configurations, creating a coding environment that suits your style and comfort.
- h. **Learning Resources and Community Support** - The popularity of VS Code means there are countless tutorials, forums, and documentation available online. As a beginner, you'll never feel stuck since the community is always ready to help.

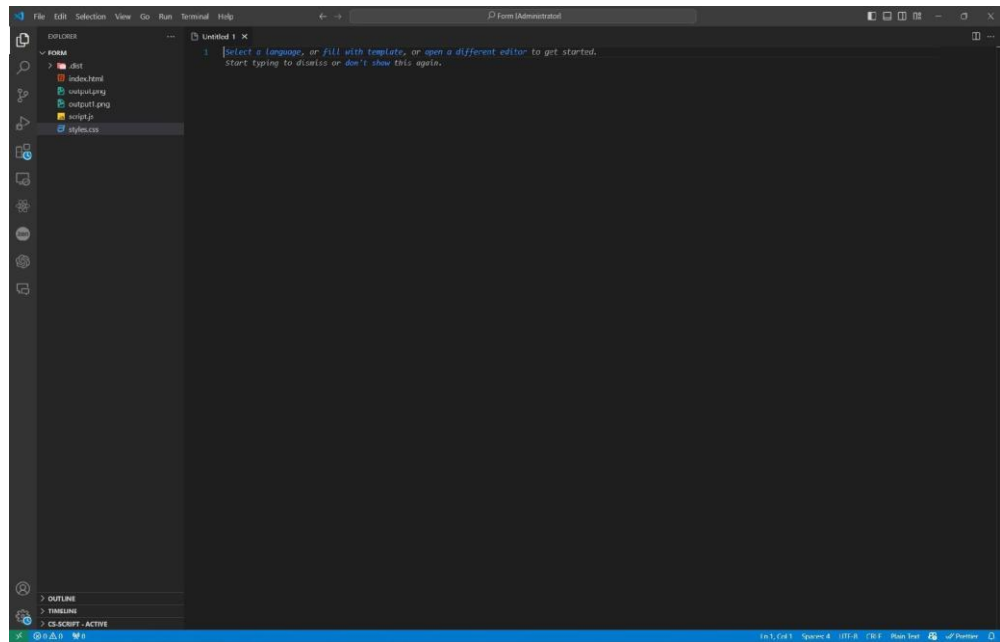


Figure 42: VS Code

2. Sublime Text

Sublime Text is a sleek, lightweight, and highly efficient text editor, widely regarded as a go-to tool for beginners stepping into programming. Known for its speed, simplicity, and ease of use, Sublime Text provides a distraction-free environment that is perfect for writing and managing code.

Key Features of Sublime Text

- a. **"Goto Anything" for Quick Navigation** - Sublime Text's "Goto Anything" feature allows you to jump to files, lines, or specific symbols within a project instantly. By simply typing part of a file name or function, you can access what you need without wasting time, making navigation effortless.
- b. **Multiple Selections and Editing**- One of the standout features of Sublime Text is its Multiple Selections capability. This allows you to select and edit multiple lines or instances of text at once, saving time and making repetitive tasks like renaming variables incredibly simple.
- c. **Command Palette for Easy Function Access**- The Command Palette is a versatile tool that provides quick access to various commands and functions without navigating menus. Whether you need to change settings, sort lines, or install a plugin, it's all just a keystroke away.
- d. **Distraction-Free Mode**- Sublime Text's Distraction-Free Mode removes all UI elements, leaving only the code on the screen. This clean and focused interface is ideal for beginners

who want to concentrate on coding without unnecessary distractions.

- e. **Split Editing** - With Split Editing, you can open and view multiple files or sections of a single file side by side. This is especially helpful when working on related code files or referencing documentation while writing code.
- f. **Extensibility with Packages** - While Sublime Text is lightweight and minimalist by default, you can enhance its functionality using packages and plugins. The Package Control system lets you browse and install extensions, such as linters, debuggers, and language-specific tools.
- g. **Cross-Platform Support** - Sublime Text is available on Windows, macOS, and Linux, providing a consistent experience across operating systems. This makes it a flexible choice for beginners who might work on different platforms.
- h. **Syntax Highlighting and Code Snippets** - Sublime Text supports syntax highlighting for a wide range of programming languages, making code more readable. It also comes with built-in code snippets that help beginners write boilerplate code faster.

Advantages of Sublime Text for Beginners

- i. Sublime Text is incredibly fast, even with large files. It launches almost instantly and runs smoothly on most systems, making it ideal for beginners with older or less powerful computers.
- ii. Unlike more complex editors or IDEs, Sublime Text offers a minimalist design that is easy to learn and navigate, making it perfect for those just starting their programming journey.
- iii. Beginners can customize Sublime Text to suit their preferences, from color schemes and themes to keyboard shortcuts. This level of customization ensures a comfortable coding environment.
- iv. While editors like Visual Studio Code or Atom might feel overwhelming with their extensive features, Sublime Text's simplicity ensures that beginners are not bogged down by unnecessary options.
- v. Sublime Text does not require an internet connection for most of its functionality, allowing beginners to practice coding anywhere.

Drawbacks

- vi. **Limited Built-In Features:** Sublime Text lacks integrated tools like Git support, debugging, or live server options found in editors like VS Code. These can be added via packages, but may require additional setup.
- vii. **Paid License:** While Sublime Text is free to use indefinitely with occasional prompts to

purchase, a one-time license fee is required for unrestricted usage.

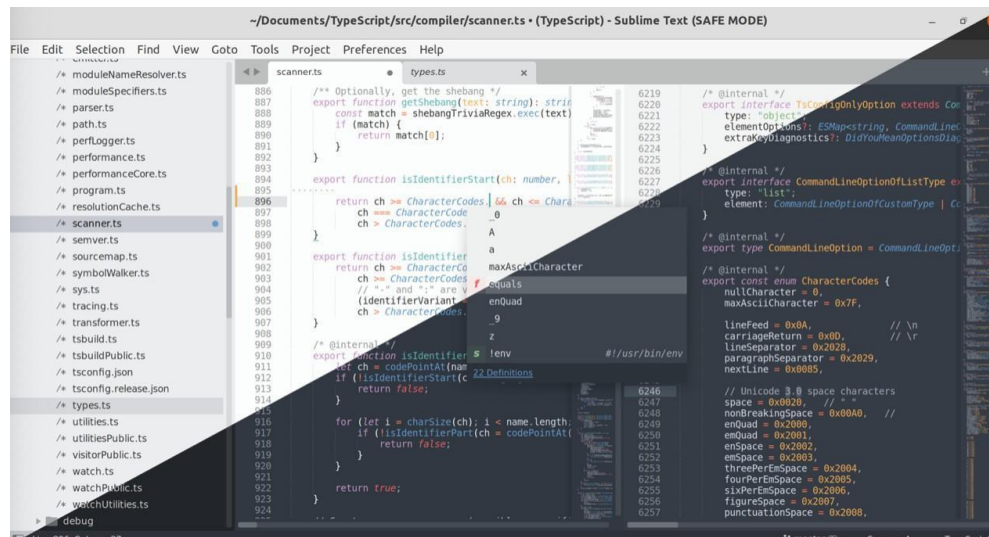


Figure 43: Sublime Text

- viii. **Smaller Ecosystem:** Sublime Text has fewer plugins and community resources compared to editors like VS Code or Atom.

3. Atom

Atom is a free and open-source text editor developed by GitHub, widely regarded for its simplicity, collaborative features, and robust customization options. Designed with beginners in mind, Atom is an excellent tool for those stepping into programming and looking for a supportive environment to grow their skills.

Key Features of Atom

- a. **Teletype: Real-Time Collaboration** - Atom's standout feature, Teletype, enables real-time collaboration by allowing multiple users to work on the same code-base simultaneously. This is particularly beneficial for group learning, coding projects, and mentorship sessions, making Atom a social and engaging platform for beginners.
- b. **Seamless GitHub Integration** - Since Atom is developed by GitHub, it integrates effortlessly with Git and GitHub. This lets beginners practice version control—an essential skill in software development—without needing to leave the editor. You can stage, commit, and push changes directly, simplifying the learning curve for managing code repositories.
- c. **Customizability** - Atom is highly customizable, allowing users to tweak nearly every aspect of the editor to suit their preferences. Beginners can explore a wide range of themes, plugins, and configurations to create a workspace tailored to their needs. Whether you want a dark theme for late-night coding or an extension for language-specific support, Atom has you

covered.

- d. **Package Ecosystem** - Atom has a vast library of community-developed packages that enhance its functionality. Beginners can install packages to add linters, formatters, debuggers, or support for additional programming languages. Popular packages like platformio-ide-terminal, emmet, and atom-beautify help simplify coding tasks and improve productivity.
- e. **Cross-Platform Compatibility** - Atom runs smoothly on Windows, macOS, and Linux, ensuring that beginners can use it regardless of their operating system. Its consistent interface across platforms ensures a seamless experience when switching devices.
- f. **Multi-Language Support** - Atom supports multiple programming languages out of the box, making it a versatile choice for beginners exploring different areas of coding, from web development to scripting and more.
- g. **Simple Learning Curve** - Atom's interface is clean and beginner-friendly, making it easy to navigate even for those with little to no prior coding experience. The editor includes intuitive features like a file tree view, fuzzy file finder, and syntax highlighting.

Advantages of Atom Compared to Other Editors

- i. Atom's real-time collaboration features set it apart from many other text editors, making it ideal for team projects and group learning.
- ii. While editors like Sublime Text offer customization, Atom excels with its open-source nature and extensive library of free packages, giving beginners endless possibilities to modify their coding environment.
- iii. Unlike Visual Studio Code, where Git integration is robust but may seem advanced for some beginners, Atom's GitHub-centric design simplifies version control for first-time users.
- iv. Unlike some editors or IDEs that come with paid versions, Atom is entirely free and open-source, ensuring accessibility for everyone.
- v. Being developed by GitHub and having an active community, Atom provides a wealth of tutorials, forums, and documentation to support beginners in their learning journey.

Drawbacks to Consider

While Atom offers many advantages, there are a few aspects to keep in mind:

- vi. **Performance:** Atom can be slower compared to lightweight editors like Sublime Text or faster options like Visual Studio Code, especially when working on large projects.
- vii. **No Built-In Debugger:** Unlike IDEs or even VS Code, Atom does not come with a built-in

debugger. Beginners may need to rely on external tools or packages for debugging.

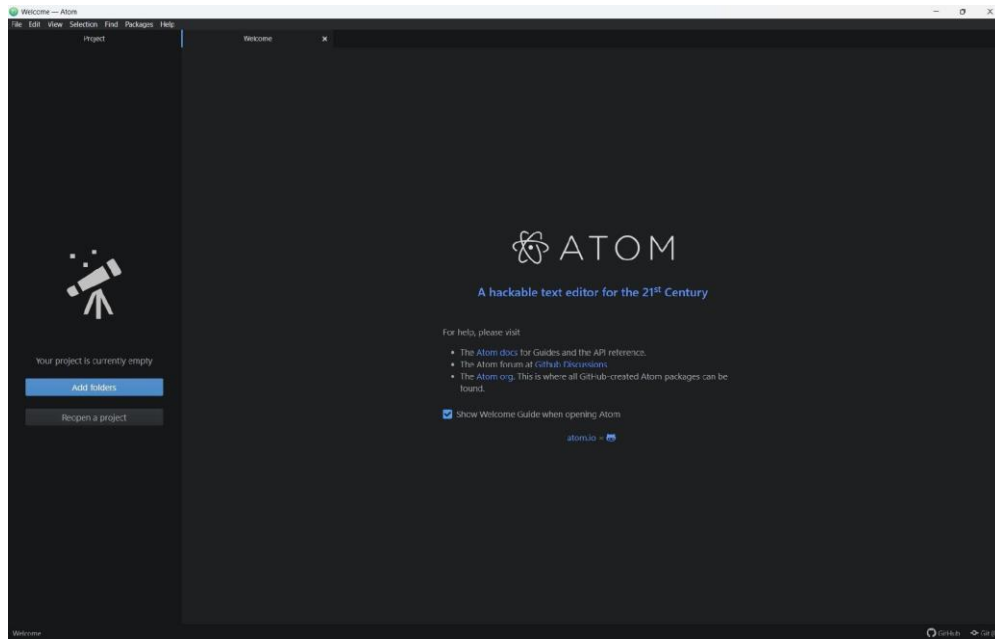


Figure 44: Atom

MODEL VIVA QUESTIONS

1. What is a repository in Git?
2. What is the difference between a local and a remote repository?
3. What does git init do?
4. Explain the purpose of git add and git commit.
5. What is the use of git clone?
6. How does git push differ from git pull?

RESULT

Gained hands-on experience with Visual Studio Code, Sublime Text, and Atom, understanding their features, interface, and how to use them effectively for development tasks.

EXP NO :14**Introducing Repositories - Git / Bitbucket****AIM**

To understand the concept of repositories and familiarize with version control systems like Git and Bitbucket, including basic operations such as creating, cloning, committing, and pushing changes.

LEARNING OUTCOMES

Students will learn to use version control systems to manage source code efficiently, track changes, and collaborate on projects. They will gain skills in creating repositories, committing changes, branching, merging, and pushing/pulling code using Git and Bitbucket.

DESCRIPTION

A repository (repo) is a storage space where your project's files, along with their revision history, are saved. Repositories allow you to:

1. Version Control: Track changes to your code over time.
2. Collaboration: Work with others on the same project without overwriting each other's work.
3. Branching: Experiment with new features without affecting the main project.
4. Backup: Safeguard your work by syncing it to remote repositories like GitHub or Bitbucket.

Repositories**1. Git: The Version Control System**

Git is an open-source version control system that helps track code changes and facilitates collaboration. It works both locally (on your computer) and remotely (on platforms like GitHub, GitLab, or Bitbucket).

Basic Git Commands:

- a. Initialize a Repository

```
bash
git init
```

 Copy code

- b. Add Files to Stage for Commit

```
bash Copy code  
  
git add <file_name> # Add specific file  
git add .           # Add all files
```

c. Commit Changes

```
bash Copy code  
  
git commit -m "Your commit message"
```

d. Push to a Remote Repository

```
bash Copy code  
  
git push origin main
```

e. Clone an Existing Repository

```
bash Copy code  
  
git clone <repository_url>
```

2. Bitbucket: A Remote Repository Hosting Service

Bitbucket is a web-based hosting service for repositories, developed by Atlassian. It supports Git (and Mercurial) repositories and is widely used for collaborative development.

Why Use Bitbucket?

1. Private Repositories: Free private repositories for small teams.
2. Integration with Atlassian Tools: Works seamlessly with Jira, Trello, and Confluence.
3. Pull Requests: Facilitate code reviews and collaboration before merging changes.
4. Pipelines: Automate testing and deployment with built-in CI/CD features.

Getting Started with Bitbucket:

- a. Create an Account: Sign up at [Bitbucket.org](https://bitbucket.org).
- b. Create a Repository:
 - i. Navigate to your dashboard and click "Create Repository."
 - ii. Choose repository type (Git), visibility (private/public), and provide a name.
- c. Clone the Repository Locally:

```
bash Copy code  
git clone <repository_url>
```

- d. Collaborate with Team Members:
 - i. Add team members to your repository.
 - ii. Use pull requests to review and merge code changes.

MODEL VIVA QUESTIONS

1. What is Git?
2. What is a repository in Git?
3. What is the difference between Git and Bitbucket?
4. What is the purpose of the git init command?
5. What is the difference between git clone and git init?

RESULT

Successfully learned how to create and manage repositories using Git and Bitbucket, perform basic version control operations, and collaborate on projects effectively.